

Homework A

Klik en typ de subtitel

Enmin Lin, Kaixi Yao

mei 2026

As Fig. 1 shows, one 3-DOF positioning arm can exhibit one, two, or infinitely inverse position solutions, depending on the target point.

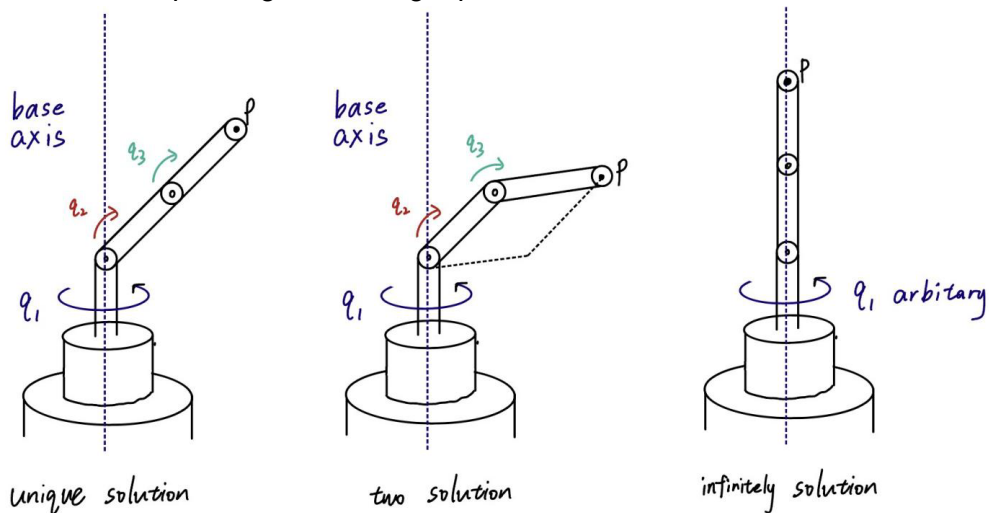


Fig. 1. Three inverse position kinematics cases for a 3-DOF positioning arm with base yaw q_1 , shoulder pitch q_2 , and elbow pitch q_3 .

1. In the left panel, the target lies on the workspace boundary. The arm is fully stretched, so the elbow-up and elbow-down branches merge into one singular configuration. This gives one inverse position solution.
2. In the middle panel, the target lies at a generic reachable point. The base yaw stays unique, and the planar two-link geometry gives the standard elbow-up and elbow-down configurations. This gives exactly two inverse position solutions.
3. In the right panel, the target lies on the base rotation axis. The base yaw does not change the end-effector position in that case. The variable q_1 is free, so infinitely many inverse position solutions exist.

2 Q2

Given initial configuration

$$q_s = (x_s, y_s, \phi_s, \theta_s)$$

and goal configuration

$$q_g = (x_g, y_g, \phi_g, \theta_g).$$

2.1 Move to the target position (x_g, y_g) .

First turn in place until the heading ϕ points from (x_s, y_s) toward (x_g, y_g) .



Then roll straight until the contact point reaches (x_g, y_g) .

This changes x, y , and also changes the rolling angle θ , but that is fine because θ will be corrected later.

2.2 Set the desired heading ϕ_g .

Once the coin is at (x_g, y_g) , turn in place until $\phi = \phi_g$.

Turning in place changes the heading but does not change the contact point position.

2.3 Correct the rolling angle θ without changing position or heading.

Now compute how much rolling angle is still missing, modulo 2π :

$$\Delta\theta = \theta_g - \theta$$

modulo 2π .

Execute the following closed maneuver:

1. roll forward by a distance L ;
2. turn in place by π ;
3. roll forward again by the same distance L ;
4. turn in place by π .

After this loop, the coin has returned to the same (x, y) and the same heading ϕ . However, it has rolled a total distance $2L$, so the rolling angle θ has changed. Choose L so that this change equals the required $\Delta\theta$ modulo 2π .

Therefore, the algorithm reaches the desired final configuration $x_g, y_g, \phi_g, \theta_g$.

2.4 Why this works

The key geometric property is that the non-holonomic constraints forbid sideways instantaneous motion, but they do not remove configurations from the C-space. By combining allowed motions, especially closed loops, we can create net changes in variables that cannot be changed independently at one instant. This is the same idea as parking a car: it cannot move sideways instantly, but it can still reach sideways-displaced configurations through forward/backward and turning maneuvers.

3 Q3



$$\text{tr}(R) = 1 + 2 \cos \theta$$

From Q3:

$$R = \begin{bmatrix} 0 & 0 & -1 \\ -1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

First, calculate the trace:

$$\text{tr}(R) = 0$$

so

$$0 = 1 + 2 \cos \theta, \cos \theta = -\frac{1}{2}$$

Select the principal angle:

$$\theta = \frac{2\pi}{3}$$

Since $\sin \theta \neq 0$, we can use the matrix logarithm formula:

$$\hat{\omega} = \frac{R - R^T}{2 \sin \theta}$$

Calculating

$$R - R^T = \begin{bmatrix} 0 & 1 & -1 \\ -1 & 0 & -1 \\ 1 & 1 & 0 \end{bmatrix}$$

Since

$$2 \sin \frac{2\pi}{3} = \sqrt{3}$$

So

$$\hat{\omega} = \frac{1}{\sqrt{3}} \begin{bmatrix} 0 & 1 & -1 \\ -1 & 0 & -1 \\ 1 & 1 & 0 \end{bmatrix}$$

Compare it with the standard skew-symmetric matrix:

$$\hat{\omega} = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}$$

Then get

$$\omega = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix}$$

Therefore, exponential coordinates are

$$\omega \theta = \frac{2\pi}{3} \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix} = \begin{bmatrix} \frac{2\pi}{3\sqrt{3}} \\ -\frac{2\pi}{3\sqrt{3}} \\ -\frac{2\pi}{3\sqrt{3}} \end{bmatrix}$$

Matrix form is

$$\hat{\omega} \theta = \frac{2\pi}{3\sqrt{3}} \begin{bmatrix} 0 & 1 & -1 \\ -1 & 0 & -1 \\ 1 & 1 & 0 \end{bmatrix}$$

Note: exponential coordinates are not unique, because rotations can be represented by equivalent axis-angles; the above is the principal solution. The pair $(-\omega, -\theta)$ represents the same rotation, so the equivalent answer also exist.



4 Q4

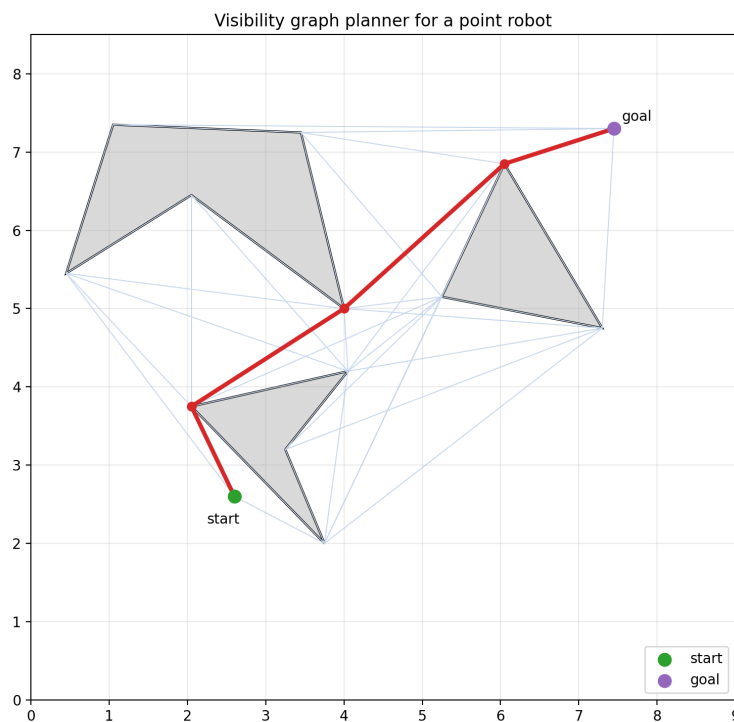


Fig. 2. Visibility-graph path for the point robot in the workspace of Homework A.

I implement a complete path planner for a point robot among polygonal obstacles using a visibility graph, following the method from Lecture 5.

The configuration space is the 2D space, since the robot is a point robot. The obstacles are polygonal regions. I chose the start, goal, and obstacle vertices to approximately match the figure in the assignment.

The planner works as follows:

1. Add the start point, goal point, and all obstacle vertices as graph nodes.
2. For every pair of nodes, check whether the straight-line segment between them intersects any obstacle.
3. If the segment is collision-free, add an edge to the visibility graph with Euclidean distance as cost.
4. Run A* search on this graph, using Euclidean distance to the goal as the heuristic.
5. Return the shortest collision-free path found.

The obstacle vertices used in my implementation are:

start = (2.60, 2.60)

goal = (7.45, 7.30)

obstacle 1: [(0.45, 5.45), (1.05, 7.35), (3.45, 7.25), (4.0, 5.0), (2.05, 6.45)]

obstacle 2: [(5.25, 5.15), (6.05, 6.85), (7.3, 4.75)]

obstacle 3: [(2.05, 3.75), (4.05, 4.2), (3.25, 3.2), (3.75, 2.0)]

The computed shortest path is:

(2.60, 2.60)

(2.05, 3.75)

(4.00, 5.00)

(6.05, 6.85)

(7.45, 7.30)

With total length: 7.822886

This planner is complete for this setting because, for a point robot with polygonal obstacles, a shortest collision-free path consists of straight-line segments whose turning points occur at obstacle vertices. Therefore, the visibility graph contains the optimal path candidate, and A* finds the shortest path in that graph.